國立勤益科技大學資訊工程系學生實務專題成果報告

※※※※※※※※※※※※※※※※※※※※※※※※※
※　　※
※　　影像姿勢與肌電訊號的健身輔助系統　　※
※　　※
※※※※※※※※※※※※※※※※※※※※※※※※※

執 行 期 間 ：113 學年度上學期至 113 學年度下學期

專題參與人員：

部別：日間部

組別：88

班別：四訊四丙 座號：58 姓名：余芃節

指導老師：陳明德教授

中　華　民　國　113 年 5 月 21 日

目錄

影像姿勢與肌電訊號的健身輔助系統 7
一、 前言 8
二、 專題簡介 9
(一) 製作目的 9
(二) 製作方法 9
(三) 製作結果 9
三、 製作理論探討 10
(一) 影像輸入到 Pose 推論流程 10
啟動相機&綁定 Analyzer 11
Analyzer 核心 20 行 11
processPose()任務切分 11
FPS 計算(效能度量) 12
小結 12
(二) 姿勢辨識相關理論與公式推導 12
夾角核心計算程式 13
角度映射到進度條(%) 13
Rep/Set 邏輯（以「二頭彎舉 curl」為例） 13
小結 14
(三) 骨架繪製 14
相機到螢幕座標縮放 14
單點繪製紅點 15
骨架連線 15
(四) EMG 原理與力竭判定 16
什麼是肌電信號（EMG）？ 16
短時均方根（RMS）計算 16
基線（MVC）校正 16

疲勞判定邏輯 16
為什麼這樣設計？ 17
關鍵常數 17
MVC 建立+RMS 計算核心(節錄) 17
警報與紀錄 18
Arduino 感測器流程 18
(五) 藍芽低功號協定棧 19
什麼是 BLE？ 19
BLE 連線流程 20
固定 UUID 常數 20
連線點呼叫 21
GATT 連線完整流程 21
精簡核心程式碼 22
Notification 應用 22
動態 CallBack 切換 22
(六) 訓練歷史查詢 23
資料表結構 23
HistoryActivity 核心互動 23
(七) 資料庫 24
(八) 雲平台架設與遠端連線 25
XAMPP 架設本地後端 25
Tailscale 實現外網存取 25
優勢與注意事項 25
手機上傳流程圖 25
核心程式 26
參數對照表 26
錯誤處理&使用者回饋 27
伺服器端上傳介面 27

(九) App 整體運作流程圖與架構圖 28
高階系統架構圖 28
影像辨識模組架構圖 29
疲勞偵測模組架構 29
雲平台連線架構圖 30
使用者流程圖 30
四、 軟硬體分析 30
(一) 軟體分析: 31
開發環境與工具 31
系統架構與模組 31
(二) 硬體／電路分析 41
主要元件清單 41
電路示意圖 43
資料與電力流程 43
五、 研究結果 45
主觀滿意度問卷（Likert 5 分量表） 45
自動化動作計數效果 47
專業教練回饋 47
六、 結論 49
七、 參考文獻 50

圖目錄

圖 1 影像辨識模組流程圖 10
圖 2 啟動相機&綁定 Analyzer 程式碼 11
圖 3 Analyzer 核心 20 行程式碼 11
圖 4 影像辨識分切任務程式碼 12
圖 5 計算 FPS 程式碼 12
圖 6 計算三點夾角公式 12
圖 7 夾角核心計算程式碼 13
圖 8 角度映射到進度條程式碼 13
圖 9 二頭彎舉計數程式碼 14
圖 10 相機到螢幕縮放程式碼 14
圖 11 骨架單點繪製程式碼 15
圖 12 骨架連線程式碼 15
圖 13 RMS 計算公式 16
圖 14 力竭判斷核心程式瑪 18
圖 15 力竭判斷警報程式碼 18
圖 16 Arduino 硬體程式碼 19
圖 17 BLE 連線圖 20
圖 18 連線點呼叫程式碼 21
圖 19 GATT 連線流程 21
圖 20 藍芽連線程式碼 22
圖 21 Notification 程式碼 22
圖 22 CallBack 程式碼 22
圖 23 歷史紀錄頁面核心互動程式碼 23
圖 24 資料表 24
圖 25 手機上傳流程圖 25
圖 26 手機上傳核心程式碼 26
圖 27 伺服器上傳介面程式碼 28
圖 28 高階系統架構圖 29
圖 29 影像辨識模組架構圖 29
圖 30 疲勞偵測模組架構圖 29
圖 31 雲平台架構圖 30
圖 32 使用者流程圖 30
圖 33 手機 App UI 進入流程 31
圖 34 HomeActivity 32
圖 35 ParticipantActivity 32
圖 36 CloudPendingActivity 33

圖 37 SettingActivity 33
圖 38 MovementSelectActivity 34
圖 39 InstructionActitivy 34
圖 40 CameraFirstActivity 35
圖 41 HistoryActivity 35
圖 42 FatigueSettingActivity 36
圖 43 BluetoothActicity 36
圖 44 DeviceTestActivity 37
圖 45 RmsChartActivity 37
圖 46 影像偵測模組流程圖 38
圖 47 EMG 處理模組流程圖 38
圖 48 本地資料存取 39
圖 49 雲端資料存取 39
圖 50 雲平台選取受試者 40
圖 51 雲平台 BMI 曲線 40
圖 52 雲平台訓練容量 41
圖 53 雲平台訓練週期 41
圖 54 Arduino Uno 3 42
圖 55 Grove EMG Detector 42
圖 56 HM-10 42
圖 57 2C LCD 1602a 42
圖 58 samsung galaxy A60 43
圖 59 電路示意圖 43
圖 60 資料與電力流程 43
圖 61 硬體元件實作展示 44
圖 62 滿意度調查生理性別 45
圖 63 滿意度調查受試者年齡 45
圖 64 滿意度調查動作精準度 46
圖 65 滿意度調查力竭警報精準度 46
圖 66 滿意度調查整體使用便利性 46
圖 67 與市面上的 App 比較 47
圖 68 教練使用裝置實測 48

表目錄

表 1 Analyzer 程式使用方法 11
表 2 關節點對應 Landmark 編號 15
表 3 力竭判斷參數說明 17
表 4 藍芽 UUID 常數 20
表 5 歷史紀錄資料庫 Table 23
表 6 資料表說明 24
表 7 傳入雲端的資料表 26
表 8 軟硬體分析圖表 30
表 9 手機 App UI 對應表格 31
表 10 硬體元件對照表 43

影像姿勢與肌電訊號的健身輔助系統

A Fitness Assistance System Integrating Vision-Based Pose Tracking and sEMG Analysis

余芃節 [1] 陳明德 [6]

Yu Peng Jie[1], Chen Ming Te[6]

國立勤益科技大學資訊工程系

Department of Computer Science and Information Engineering National Chin-Yi University of Technology

摘要

本專題旨在開發一套結合影像辨識與肌電訊號（EMG）的智慧健身輔助系統，解決一般使用者在居家或健身房訓練時，容易忽略姿勢與肌肉疲勞狀態所造成的運動傷害問題。系統可透過影像自動辨識訓練動作，紀錄組數與休息時間，並藉由肌電感測裝置偵測使用者是否力竭，即時發出警示，協助使用者安全有效地進行訓練，提升健身紀錄的便利性與準確性。

Abstract

This project aims to develop an intelligent fitness assistance system integrating image recognition and electromyography (EMG) signals. It addresses the issue of exercise-related injuries commonly caused by incorrect posture and muscle fatigue, which users often overlook during workouts at home or in gyms. The system automatically recognizes training movements through image recognition, recording sets and rest periods. Additionally, by using an EMG sensing device, it detects muscle fatigue and provides immediate alerts, assisting users in exercising safely and effectively, thus enhancing the convenience and accuracy of workout records.

一、 前言

隨著健康意識的抬頭與社群媒體對健身風氣的推波助瀾，越來越多民眾投入運動訓練，不論是前往健身房進行重量訓練，或在家中進行簡易的伸展與肌力訓練，健身已成為現代人維持身心健康的重要方式之一。然而，在缺乏專業指導與即時監控的情況下，許多使用者容易忽略肌肉負荷程度，導致運動傷害、過度訓練甚至肌肉溶解等嚴重後果。

傳統的健身應用程式多仰賴使用者手動輸入訓練組數與動作完成狀況，操作繁瑣且無法即時提供使用者肌肉狀況的回饋，難以有效預防潛在的傷害風險。單靠影片教學或線上課程學習正確姿勢，也無法確保使用者實際執行的準確度與安全性。因此，如何建構一套能主動偵測姿勢與肌肉狀態、並即時反饋的智慧健身系統，成為本專題的研究動機。

本專題結合影像辨識與肌電訊號（EMG）技術，設計並實作一套智慧健身輔助系統，透過鏡頭即時分析使用者動作姿勢並自動記錄訓練過程，同時藉由感測器即時偵測肌肉活動變化，當判斷使用者已達力竭狀態時發出提示或警示，以提升訓練的安全性與便利性，並協助使用者建立更有效率的運動習慣。

二、　專題簡介

(一)製作目的

1. 解決一般健身族群無法即時掌握自身動作姿勢與肌肉狀態的問題。
2. 降低因姿勢不當或過度訓練所導致的運動傷害風險。
3. 提供一套自動化、智慧化的健身輔助系統，減少使用者手動操作負擔。
4. 結合硬體與軟體技術，提升居家與健身房訓練的安全性與效率。

(二)製作方法

1. Pose 偵測：使用 MediaPipe Pose 擷取人體 33 個關鍵點。
2. 動作計數：計算指定三點（如肩－肘－腕）之夾角變化，自動辨識動作次數與組數。
3. EMG 收集：Arduino Uno 結合 Grove EMG 模組與 HM-10 藍牙模組讀取肌電訊號。
4. 力竭判定：在行動裝置上當正規化後的 RMS 值連續低於閾值（0.6）達到 5 次時，就判定為力竭狀態。
5. App 開發：Android Studio + Java，整合 CameraX、ML Kit、BLE、SQLite。
6. 本地資料庫：SQLite 儲存運動紀錄、休息時間、完成時間等。
7. 雲端後端：XAMPP（PHP + MySQL）架設 API，使用 Retrofit 串接並上傳待處理資料（CloudPendingActivity）。
8. 資料可視化：網頁端介面提供雲端資料瀏覽與圖表分析。

(三)製作結果

1. 即時力竭提示：App 可在使用者肌肉力竭當下發出提醒，實現安全回饋。
2. 自動動作計數：自動記錄組數與次數，減少手動登錄錯誤。
3. 雲端資料同步：訓練資料成功上傳至 MySQL，網頁端可即時查詢與趨勢分析。
4. 歷史追蹤：提供多日訓練記錄篩選與雲平台圖表呈現，便於長期效果評估。

三、 製作理論探討

(一)影像輸入到 Pose 推論流程

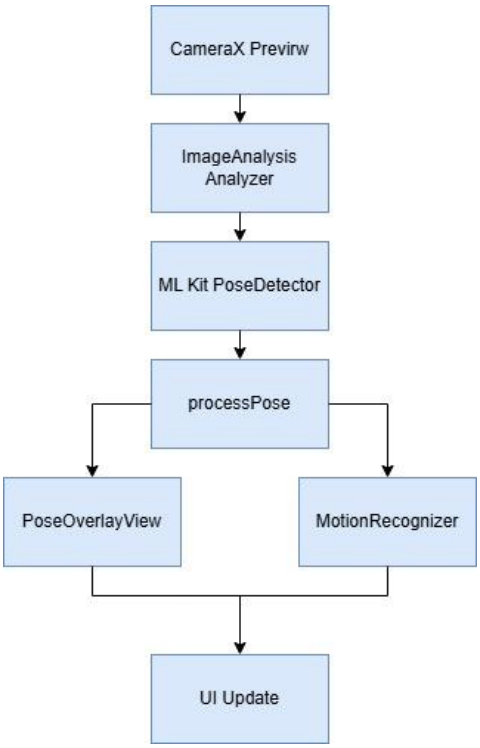


圖 1 影像辨識模組流程圖

1. CameraX Preview：想像它就是手機鏡頭的預覽畫面，畫面進來後，CameraX 幫你把影像「接好、顯示好」。
2. ImageAnalysis Analyzer：它是個「影格過濾器」，每幀影像都丟給它，它只保留最新一幀然後傳給下一步，確保不會卡住或堆積老舊畫面。
3. ML Kit PoseDetector：這個模組拿到影像就開始「腦力運算」，幫你找出人體的 33 個關鍵點（肩膀、手肘、手腕⋯⋯），輸出一個 Pose 物件。
4. processPose()：把剛才偵測到的 Pose 交給這裡，做兩件事：
   I. 一邊交給骨架畫圖的元件（PoseOverlayView）畫紅點綠線
   II. 另一邊交給動作計數元件（MotionRecognizer）算你做了幾下、做了幾組
5. PoseOverlayView：負責把骨架點和骨架線畫到畫面上，如果你在用前鏡頭，還會自動做左右翻轉校正，讓左手永遠出現在畫面的左邊。
6. MotionRecognizer：它拿到每一幀關鍵點的座標，用三點夾角演算法判斷「手臂抬高到哪裡？放下到哪裡？」然後決定何時加１下、何時加１組，還計算目前的角度進度（0–100％）。
7. UI Update：最後兩個模組的結果（骨架畫好、次數/組數/進度有新數值）都送回主線程，用 runOnUiThread() 把畫面上的次數、組數文字和進度條更新成最新狀態。

啟動相機&綁定 Analyzer

```
imageAnalysis = new ImageAnalysis.Builder()
        .setTargetAspectRatio(AspectRatio.RATIO_16_9)   // 固定 16:9，避免動態裁切
        .setBackpressureStrategy(ImageAnalysis.STRATEGY_KEEP_ONLY_LATEST)
        .build();

imageAnalysis.setAnalyzer(MainExecutor, this);  // ← CameraFirstActivity implements Analyzer
```

圖 2 啟動相機&綁定 Analyzer 程式碼

Backpressure 揀最新影格：確保推理來不及時捨棄舊 Frame，降低延遲。

MainExecutor：使用 UI 執行緒即可，ML Kit 內部另開 thread。

Analyzer 核心 20 行

```
@Override
public void analyze(@NonNull ImageProxy proxy){
    // 1. FPS 基準   (每秒重設 frameCount)
    frameCount++; updateFpsIfNeeded();

    Image media = proxy.getImage();
    if (media == null){ proxy.close(); return; }

    // 2. YUV → InputImage，並帶入感測器角度
    InputImage img = InputImage.fromMediaImage(
            media,
            proxy.getImageInfo().getRotationDegrees());

    // 3. 非同步推理
    poseDetector.process(img)
        .addOnSuccessListener(p -> {              // p = Pose 結果
            processPose(p, proxy.getWidth(), proxy.getHeight());
            proxy.close();                        // *一定*要關，否則 Preview 卡住
        })
        .addOnFailureListener(e -> {
            Log.e("Camera", "pose fail", e);
            proxy.close();
        });
}
```

圖 3 Analyzer 核心 20 行程式碼

表 1 Analyzer 程式使用方法

步驟	目的	注意點
getImage()	拿原生 YUV420 影格	可能為 null，務必檢查避免 NPE
fromMediaImage()	把旋轉角度一併包進 InputImage	ML Lit 圖像座標系統依此校正
process()	GPU or NNAPI 自動選擇	
proxy.close()	回收 buffer	少掉這行會導致畫面凍結、記憶體外洩

processPose()任務切分

```java
boolean isFrontCam = (cameraFacing == CameraSelector.LENS_FACING_FRONT); // ← 判斷鏡頭方向

// 1) 畫骨架 (若前鏡頭 → PoseOverlayView 內部做 x 翻轉)
poseOverlayView.updatePose(pose, screenW, screenH,
        imgW, imgH, isFrontCam);

// 2) 動作計數
motionRecognizer.updatePose(pose);

// 3) 取最新 rep / set / 角度% 並更新 UI
int rep  = motionRecognizer.getCurrentRepCount();
int set  = motionRecognizer.getCurrentSetCount();
int prog = motionRecognizer.getAngleProgress();
runOnUiThread(() -> {
    tvCurrentSets.setText("已完成組: " + set);
    tvCurrentReps.setText("本組次數: " + rep);
    angleProgressBar.setProgress(prog);
});
```

圖 4 影像辨識分切任務程式碼

單一責任原則 (SRP)：CameraFirstActivity 只負責資料流調度；演算法與繪圖各自封裝。

鏡像修正：isFrontCam 旗標讓 PoseOverlayView 在前鏡頭時做 x = screenW - x 翻轉。

FPS 計算(效能度量)

```java
if (now - lastFpsTime >= 1000){
    currentFps = frameCount;  // 近 1 秒影格數 = FPS
    frameCount = 0; lastFpsTime = now;
    runOnUiThread(() -> fpsTextView.setText("FPS:"+currentFps));
}
```

圖 5 計算 FPS 程式碼

小結

把「動作計數」與「骨架繪製」各自模組化，展示良好的可維護性。

(二)姿勢辨識相關理論與公式推導

關鍵點模型：MediaPipe Pose 提供 33 個 Landmark，包括肩膀、手肘、手腕等。

極角差值法：計算三點夾角 θ：

$$BA = (x_A - x_B, y_A - y_B), \quad BC = (x_C - x_B, y_C - y_B)$$
$$\phi_1 = \operatorname{atan2}(y_C - y_B, x_C - x_B), \quad \phi_2 = \operatorname{atan2}(y_A - y_B, x_A - x_B)$$
$$\theta = |\phi_1 - \phi_2|, \quad \theta_{final} = \begin{cases} \theta, & \theta \le 180^\circ \\ 360^\circ - \theta, & \theta > 180^\circ \end{cases}$$

圖 6 計算三點夾角公式

夾角核心計算程式

```
// === getAngle() — 計算 ∠θ (shoulder-elbow-wrist) ===
// 1. 建兩個向量：v1 = 肘→肩，v2 = 肘→腕
double x1 = first.x - mid.x;
double y1 = first.y - mid.y;
double x2 = last.x  - mid.x;
double y2 = last.y  - mid.y;

// 2. 取兩向量極角差，再轉成角度制
//    θ = |atan2(y2,x2) - atan2(y1,x1)| (°)
double angle = Math.toDegrees(Math.atan2(y2,x2) - Math.atan2(y1,x1));
angle = Math.abs(angle);

// 3. 轉成 0-180° 區間 (>180° 時取 360-θ)
if (angle > 180) angle = 360.0 - angle;
return angle;            // ← 供後續判斷
```

圖 7 夾角核心計算程式碼

為何選 0－180˚？

人體關節實際活動範圍落在 0－180˚，如此可避免拍攝鏡像或關節翻轉造成的負角度問題。

角度映射到進度條(%)

```
// === calcAngleProgress() — 夾角 → 0~100% ===
//   ratio = (θ - θ_min) / (θ_max - θ_min)
//   超界時以 0/1 夾制 (clamp)
int calcAngleProgress(double θ, double min, double max){
    double r = (θ - min) / (max - min);
    r = Math.max(0, Math.min(1, r));   // clamp 0~1
    return (int)(r * 100);             // 回傳百分比
}
```

圖 8 角度映射到進度條程式碼

用途：在 UI 端畫進度環（progress ring），使用者能即時看到「這一下已完成幾 %」。

人因設定：θ_min / θ_max 取自實測平均（如二頭彎舉 50˚-140˚），避免手臂未完全伸直也能被偵測。

Rep/Set 邏輯（以「二頭彎舉 curl」為例）

```
// === handleBicepCurl() 簡化 ===
if (!isUp && θ > 140)      // 手臂彎到最上緣 → 「上」區間
    isUp = true;           // 狀態機: down→up
else if (isUp && θ < 50){ // 回到「下」區間
    isUp = false;
    rep++;                 // 完成 1 Rep
    if (rep >= targetRep){
        set++;             // Rep 滿 → 1 Set
        rep = 0;           // 重新計次
    }
}
```

圖 9 二頭彎舉計數程式碼

有限狀態機 (FSM)：isUp=false→true→false 形成一個閉環，保證只有「一次完整上-下」才計一次 Rep，大幅減少抖動誤判。

小結

MotionRecognizer 透過 三點夾角 + FSM 即可支援多動作（curl/raise/press），且 動作閾值與 進度映射 皆封裝在專用方法內，未來要新增深蹲、硬舉只需：

1. 換三點 (hip-knee-ankle)；
2. 調 θ_min / θ_max / upThreshold / downThreshold。

此設計模組化、易維護，符合「前端運算 + 後端統計」之分層原則。

(三)骨架繪製

座標映射：計算影像—螢幕縮放係數，將 Landmark 原始座標乘上 scaleX, scaleY。

鏡像校正：前鏡頭時對 x 軸做翻轉：x_mirrored = screenWidth - x_scaled。

繪製：Canvas API drawCircle 標記關鍵點；drawLine 連接骨架邊對。

相機到螢幕座標縮放

```
// 影像尺寸(imageWidth × imageHeight) 與 預覽尺寸(previewW × previewH)
scaleX = (float) previewWidth  / imageHeight; // 注意: CameraX 影像為 90° 旋轉
scaleY = (float) previewHeight / imageWidth;
```

圖 10 相機到螢幕縮放程式碼

為何 imageHeight 作分母?

CameraX 的 ImageProxy 於 portrait 模式下先旋轉 90°，故 影像寬高與預覽寬高對調；以高度對寬、寬對高計算才能對齊。

單點繪製紅點

```
for (PoseLandmark lm : pose.getAllPoseLandmarks()){
    PointF p = lm.getPosition();      // 原影像座標 (px)
    float x = p.x * scaleX;           // → 螢幕座標
    float y = p.y * scaleY;
    if (isFrontCamera) x = previewWidth - x;  // 鏡像修正
    canvas.drawCircle(x, y, 7, pointPaint);
}
```

圖 11 骨架單點繪製程式碼

鏡像公式：x' = previewW − x，y 不變，確保左/右肢在自拍鏡頭下仍對應正確。

骨架連線

```
connect(canvas, paint, LEFT_SHOULDER, RIGHT_SHOULDER);  // 軀幹
connect(canvas, paint, LEFT_SHOULDER, LEFT_ELBOW);      // 左上臂
...
```

圖 12 骨架連線程式碼

表 2 關節點對應 Landmark 編號

部位	Landmark 編號
肩膀	11/12(L/R)
手肘	13/14
手腕	15/16
髖	23/24
膝	25/26
腳踝	27/28

(四)EMG 原理與力竭判定

什麼是肌電信號（EMG）？

想像肌肉在收縮時會產生微小的電壓變化，這就像手機在工作時會有電流流動，我們用 EMG 感測器「偷聽」這些電流波動，得到一串數字訊號。

短時均方根（RMS）計算

滑動視窗：每隔 200 個樣本（約 100 毫秒），把這段時間的訊號平方、平均，再開根號，得到一個代表肌肉「用力強度」的單一數值。

為什麼用 RMS? 它能平滑瞬間尖峰，讓我們看到整體用力趨勢，就像把河流湍急與平緩的水流都平均下來。

$$\mathrm{RMS}[k] = \sqrt{\frac{1}{N}\sum_{i=0}^{N-1} x^2[k-i]}$$

圖 13 RMS 計算公式

基線（MVC）校正

在正式偵測前，我們先記錄使用者前 100 筆數據（約 50 毫秒），找出最大值當作最大自我舒適用力（MVC），後續的疲勞判定都會以此為基準，避免每個人用力狀況不同而判斷錯誤。

疲勞判定邏輯

1. MVC 基線建立

   蒐集 100 筆 EMG 訊號，取其中最大值作為 mvcBaseline。

2. RMS 正規化計算

   每收到一筆新的 EMG，先維護一個 200 筆長度的滑動視窗，計算該視窗內的 RMS：

   rms = sqrt(sum(v_i^2) / 200)

   norm = rms / mvcBaseline

3. 連續低於閥值累計

   若 norm < 0.6，則 consecutiveCount++；否則：

   consecutiveCount = 0

   hasFatigued = false

4. 觸發力竭警示

   當 consecutiveCount >= 5 且 距上一次警示時間超過 10 秒（COOLDOWN_MS）時：

   標記 hasFatigued = true

   播放嗶聲 + 寫入資料庫

   呼叫 listener.onFatigue()

   重置 consecutiveCount = 0，更新警示時間戳

5. UI 即時回報

每次計算出 norm（且 mvcBaseline 已算出）都會呼叫 listener.onRmsUpdated(norm)，讓介面顯示當前肌肉負荷比值。

為什麼這樣設計？

貼近使用者感受：把電壓數字轉成「忙碌程度」，讓普通使用者也能一聽就懂「你現在太累了」。

兼顧效率與準確度：200 樣本的視窗剛好能兼顧即時反應（100 毫秒上下）與數據穩定，不會因為肌肉微抖就亂警報。

關鍵常數

表 3 力竭判斷參數說明

名稱	值	說明
MVC_WINDOW	100	建立 MVC 基線的樣本數
RMS_WINDOW	200	RMS 計算視窗(約 100ms)
FATIGUE_THRESHOLD	0.6	當 norm 低於此值視為高疲勞
CONSECUTIVE_LINMIT	5	連續 5 窗皆高疲勞才觸發
COOLDOWN_MS	10000	每次警報後冷卻 10 秒

MVC 建立+RMS 計算核心(節錄)

```
// 1) 建立 MVC (取 100 筆最大值)
if (mvcQueue.size()<100){
    mvcQueue.add(emg);
    if(mvcQueue.size()==100){
        mvcBaseline = 0;
        for(int v:mvcQueue) if(v>mvcBaseline) mvcBaseline=v; // MVC = max
    }
    return; // 基線未完成
}

// 2) 計算短時 RMS (N = 200)
if (rmsQueue.size()==200) rmsQueue.removeFirst();
rmsQueue.add(emg);
double sum=0; for(int v:rmsQueue) sum+=v*v;
double rms = Math.sqrt(sum / rmsQueue.size());
double norm = rms / mvcBaseline;

// 3) 疲勞狀態機
if(norm<0.6) consecutiveCount++; else consecutiveCount=0;
if(!hasFatigued && consecutiveCount>=5){
    long now=System.currentTimeMillis();
    if(now-lastFatigueTs>=10000){
        lastFatigueTs=now; hasFatigued=true;
        onFatigueDetected();
        if(listener!=null) listener.onFatigue();
    }
}

if(listener!=null) listener.onRmsUpdated(norm);
```

圖 14 力竭判斷核心程式瑪

RMS 公式：RMS = (Σ v^2 / N)

歸一化強度：norm = RMS / MVC

觸發條件：norm 連續 0.5 秒低於 0.6

警報與紀錄

```
private void onFatigueDetected(){
    playBeep();                                      // 嗶聲
    dbHelper.insertFatigueEvent(startStr,endStr,dur); // DB 紀錄
    startTimestampMs = System.currentTimeMillis();    // 重設計時
    consecutiveCount = 0;
}
```

圖 15 力竭判斷警報程式碼

playBeep() 透過 MediaPlayer 播放 beep.mp3；播放完自動釋放

DB 紀錄力竭發生時間

Arduino 感測器流程

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <SoftwareSerial.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);
SoftwareSerial BT(10, 11);          // HM-10 (RX=D10, TX=D11)

const unsigned long SAMPLE_INTERVAL_MS = 5;   // 200 Hz
unsigned long lastSample = 0;

void setup() {
  Serial.begin(9600);
  BT.begin(9600);

  lcd.begin();  lcd.backlight();
  lcd.setCursor(0,0); lcd.print("EMG Monitor");
}

void loop() {
  if (millis() - lastSample >= SAMPLE_INTERVAL_MS) {
    lastSample += SAMPLE_INTERVAL_MS;

    int emg = analogRead(A0);       // 0-1023
    Serial.println(emg);            // 給 PC
    BT.println(emg);                // 給手機

    lcd.setCursor(0,1);
    lcd.print("        ");          // 清第二行
    lcd.setCursor(0,1);
    lcd.print(emg);
  }
}
```

圖 16 Arduino 硬體程式碼

RMS 計算目前已經在手機端 FatigueMonitor 完成

Arduino 程式只負責：

- 每 5 ms 採樣 EMG 數值（200 Hz）
- 傳送數值給手機（Bluetooth / Serial）
- LCD 螢幕同步顯示（輔助 debug）

嵌入 RMS 到 Arduino 不符合整體架構

- 手機端才有全套 RMS 判斷 + MVC 基線邏輯 + UI 顯示 + SQLite 紀錄
- Arduino 端只需穩定、即時地送出原始值

(五)藍芽低功號協定棧

什麼是 BLE？

Bluetooth Low Energy（BLE）是一種省電的藍牙通訊技術，適合傳輸少量、頻繁且即時的資料，比如我們的 EMG 訊號。

BLE 連線流程

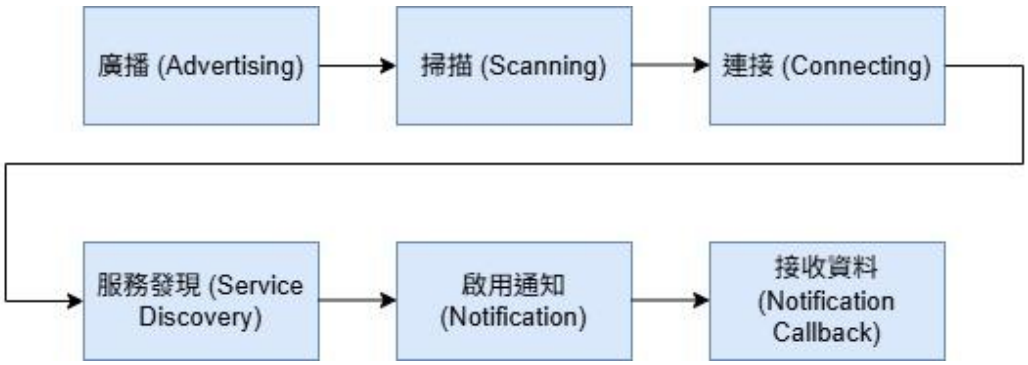


圖 17 BLE 連線圖

1. 廣播 (Advertising)：HM-10 藍牙模組定期「呼喊」自己，透過廣播包（advertising packet）宣告服務 UUID（0xFFE0），就像路口的廣告看板。
2. 掃描 (Scanning)：手機端呼叫 startScan()，像是拿著收音機搜尋指定頻道，篩選出含有 HM-10 UUID 的廣播包。
3. 連接 (Connecting)：找到目標後，用 device.connectGatt(context, false, gattCallback) 發起連線，就像按下「電話」鍵，與 HM-10 建立點對點連接。
4. 服務發現 (Service Discovery)：連線成功（onConnectionStateChange 變為 STATE_CONNECTED）後，呼叫 gatt.discoverServices()，實際拿到 HM-10 提供的所有服務清單，就像去櫃台索取路線圖。
5. 啟用通知 (Notification)：找到服務 UUID 0xFFE0 下的 characteristic UUID 0xFFE1。呼叫 gatt.setCharacteristicNotification(characteristic, true)，並寫入 CCCD descriptor（UUID 0x2902）值為 [0x01, 0x00]，告訴 HM-10「我想訂閱你的更新」。
6. 接收資料 (Notification Callback)：當 HM-10 有新的 EMG 值要推送，就會觸發 onCharacteristicChanged() 回調，手機端就能讀到 byte[]，再轉成實際電壓或 RMS 值。

固定 UUID 常數

表 4 藍芽 UUID 常數

名稱	UUID	裝置	說明
UUID_HM10_SERVICE	0000FFE0-...	HM-10	自訂 Serial Service
UUID_HM10_CHAR	0000FFE1-...	HM-10	TX/RX Characteristic
UUID_CLIENT_CHAR_CONFIG	0x2902	標準	CCCD，用於開啟 Notification

連線點呼叫

```
public void connectDevice(Context ctx, BluetoothDevice dev){
    if(bluetoothGatt!=null && isConnected) disconnect();
    bluetoothGatt = dev.connectGatt(ctx,false,gattCallback); // 自動回調 onConnectionStateChange
}
```

圖 18 連線點呼叫程式碼

Auto-Reconnect=false 失連需 UI 額外重連，避免無線重試吃電。

GATT 連線完整流程

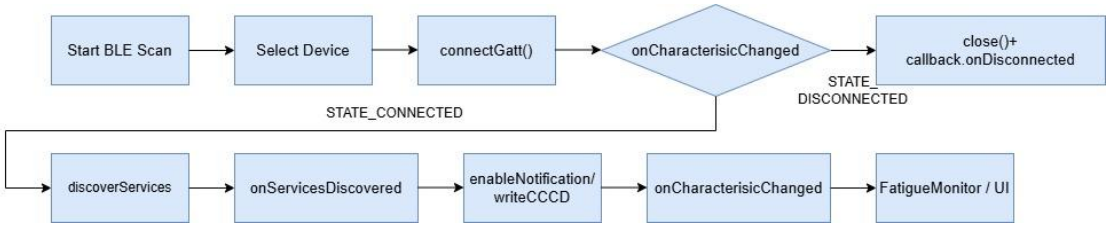


圖 19 GATT 連線流程

1. Start BLE Scan

   就像拿著手電筒在黑暗裡掃／找藍牙裝置，開始搜尋附近所有有在廣播的 HM-10。

2. Select Device

   掃到好幾台藍牙後，你點「這台我要連」，就是在列表裡選中目標裝置。

3. connectGatt()

   呼叫 device.connectGatt(...)，它開始跟那台 HM-10 建立低功耗的 GATT 連線，就像打電話給對方。

4. onConnectionStateChange → STATE_CONNECTED

   一旦連線成功，系統回報 STATE_CONNECTED，表示「電話接通了」。

5. discoverServices()

   接通後立刻問對方：「你有哪些服務？」「這些服務裡有什麼特徵值？」就是呼叫 gatt.discoverServices()。

6. onServicesDiscovered()

   系統回來說：「服務我都找到了」，觸發這個回呼，你就知道可以開始啟用通知。

7. enableNotification() / writeCCCD

   透過 setCharacteristicNotification() + 寫入 CCCD descriptor（0x2902），告訴 HM-10：「請開始主動推我資料」。

8. onCharacteristicChanged()

   每當 HM-10 有新 EMG 值，就從這個回呼拿到 byte[] data，等同於「電話裡傳來一句最新心跳值」。

9. FatigueMonitor / UI

   拿到資料後丟給 FatigueMonitor 處理（算 RMS、判定力竭），同時更新 UI（progress bar、嗶聲警示…）。

10. DISCONNECTED → close() + callback.onDisconnected

如果你主動斷開或意外斷線，就進入 DISCONNECTED，接著 gatt.close() 清理資源，並呼叫 onDisconnected 通知上層「已經真的掛斷了」。

精簡核心程式碼

```
// 1) 連線成功後 → 掃服務
if (newState == STATE_CONNECTED) {
    isConnected = true;                 // 記錄狀態
    callback.onConnected(dev);          // 通知 UI 層
    gatt.discoverServices();            // → 觸發 onServicesDiscovered()
}

// 2) 在服務探索回調中：找到 HM-10 Serial Service/Char 後開通知
BluetoothGattService svc = gatt.getService(UUID_HM10_SERVICE);
BluetoothGattCharacteristic c = svc.getCharacteristic(UUID_HM10_CHAR);
enableNotification(gatt, c);            // 實作 = setCharacteristicNotification + 寫 CCCD

// 3) 每當周邊 (HM-10) 有資料推送時 → onCharacteristicChanged()
public void onCharacteristicChanged(...){
    byte[] data = characteristic.getValue();
    callback.onDataReceived(dev, data);  // 把 raw bytes 丟給 FatigueMonitor 等解析
}
```

圖 20 藍芽連線程式碼

discoverServices()：BLE 規範要求連線後先取得服務清單，才能定位 characteristic UUID。

getService() / getCharacteristic()：根據事先知道的 HM-10 UUID 抽取 Serial 特徵值。

enableNotification()：(a) setCharacteristicNotification() 告知 Android Stack 要監聽。(b) 寫入 CCCD (0x2902) 讓周邊開始主動推送。

onCharacteristicChanged()：每次 HM-10 notify 一包資料就觸發；此處僅轉交 byte[]，解析權交給上層模組，可保持 BleManager 的純通訊職責。

Notification 應用

```
boolean ok = gatt.setCharacteristicNotification(char,true);
BluetoothGattDescriptor cccd = char.getDescriptor(UUID_CCCD);
cccd.setValue(ENABLE_NOTIFICATION_VALUE);
gatt.writeDescriptor(cccd);
```

圖 21 Notification 程式碼

兩步驟：本地 開通知 + 寫 CCCD，缺一不可。

HM-10 僅支援 Notify 無 Indicate。

動態 CallBack 切換

```
public void setCallback(BleManagerCallback cb){ this.callback = cb; }
```

圖 22 CallBack 程式碼

讓不同頁面 (Scanner / DeviceTest / FatigueMonitor) 在畫面切換時即插即拔，避免多重觀察者導

致 NullPointer 或重複處理。

(六)訓練歷史查詢

提供「依日曆篩選紀錄 → 列表顯示 → 左滑刪除」的 UI，完成資料閉環：裝置偵測 → SQLite 儲存 → 歷史回查。

資料表結構

表 5 歷史紀錄資料庫 Table

欄位	型態	說明
id	INTEGER PK	自增鍵
exercise	TEXT	動作名稱(curl/raise/press)
weight	REAL	Kg
sets	INT	組數
reps	INT	每組次數
rest_secs	INT	該組休息秒數
date	TEXT	YYYY-MM-DD
timestamp	TEXT	ISO 8601，準確到秒

HistoryActivity 核心互動

```
calendarView.setOnDateChangeListener((v,y,m,d) -> {
    String date = String.format("%04d-%02d-%02d", y, m+1, d);
    loadHistoryByDate(date);           // → DAO → Adapter
});

// 左滑刪除
new ItemTouchHelper(new SimpleCallback(0, LEFT){
    public void onSwiped(VH vh,int dir){ historyAdapter.deleteItem(vh.getAdapterPosition()); }
}).attachToRecyclerView(rvHistory);
```

圖 23 歷史紀錄頁面核心互動程式碼

即時性：日期改變 → DAO 查詢 → DiffUtil 刷新。

資料一致性：刪除後同步寫 DB + 更新 RecyclerView，避免重載。

(七)資料庫

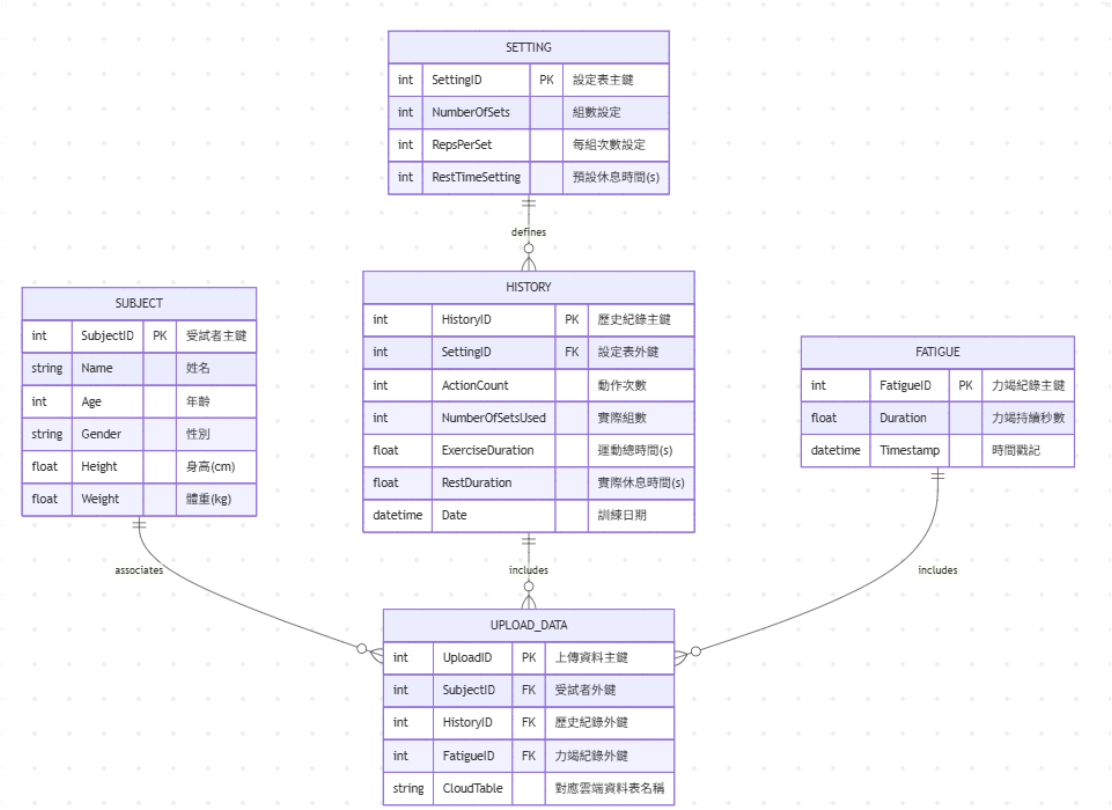


圖 24 資料表

- SETTING：紀錄運動過程的目標組數、目標次數、組間休息時間。
- SUBJECT：紀錄要上傳雲平台的受試者資訊。
- HISTORY：運動歷史紀錄。
- FATIGUE：紀錄力竭偵測開始到力竭發生的過程秒數、此次力竭發生的日期。
- UPLOAD_DATA：要上傳至雲平台的資料表，整合 SUBJECT、HISTORY、FATIGUE 表格。

手機端使用 SQLite，雲端使用 MySQL。

表 6 資料表說明

表格名稱	功能
SETTING	紀錄運動初始設定的組數、重量、休息時間等，使用者可以在 App 中做設定更改
SUBJECT	紀錄要上傳到雲平台的使用者數據
HISTORY	運動紀錄
FATIGUE	力竭紀錄
UPLOAD_DATA	合併表格為要上傳到雲平台的資料表

(八)雲平台架設與遠端連線

XAMPP 架設本地後端

在開發機或伺服器上安裝 XAMPP，包含 Apache、MySQL (MariaDB) 與 PHP。

利用 Apache 部署 PHP API：在 htdocs 建立專案資料夾，撰寫接收 POST/GET 請求的 PHP 腳本，實作資料庫 CRUD 操作。

在 MySQL 中建立運動資料表（如 users, workouts, emg_readings），設定合適欄位與索引以提升查詢效率。

使用 PHP PDO 或 MySQLi 安全連接資料庫，並透過 API 接收 App 端傳送的 JSON 資料。

Tailscale 實現外網存取

在 XAMPP 主機與開發電腦上安裝 Tailscale 客戶端，登入同一帳號並加入同個 Tailnet。

開啟 Magic DNS，透過自動配置的子域名（*.beta.tailscale.net）存取伺服器，無需設定路由器 NAT 或公開 IP。

在 Tailscale 管理介面設定 ACL（Access Control List），限制只有 App Server 與開發者端有權連線。

App 端透過 Retrofit 指定 Tailscale 提供的私有 IP 或 Magic DNS URL 呼叫 API，可從任何網路環境安全連接到本地後端。

優勢與注意事項

安全性高：Tailscale 內建 WireGuard 加密通道，避免公開伺服器暴露於公網攻擊。

部署方便：不需防火牆或路由器設定，快速完成遠端測試與示範。

可用於實機測試：讓手機 App 即使在非同一局域網，也能穩定呼叫位於實驗室或宿舍的 XAMPP 伺服器。

手機上傳流程圖

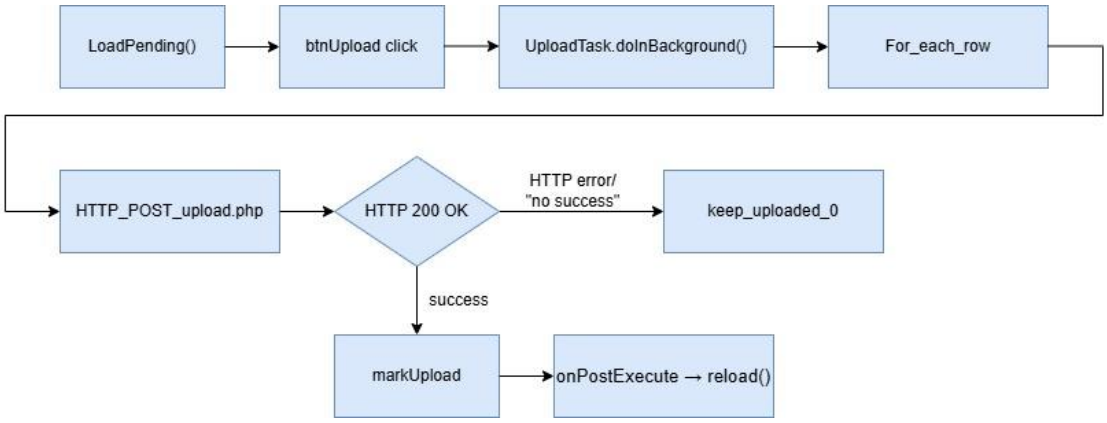


圖 25 手機上傳流程圖

1. LoadPending()：先把 SQLite 裡 uploaded=0 的待上傳記錄撈出來。

2. btnUpload click：使用者點「上傳」按鈕觸發。
3. UploadTask.doInBackground()：啟動背景執行緒，開始跑上傳迴圈。
4. for each pending row：對每筆待傳資料執行 HTTP POST。
5. HTTP POST upload.php：向 upload.php 發送 x-www-form-urlencoded 的欄位資料。
6. if success (HTTP 200 & 回傳包含 "success")：伺服器確認收到並寫入資料庫後回「success」。
   I. markUploaded：把該筆在本地 SQLite 標記 uploaded=1。
   II. keep uploaded=0：若失敗或不含 "success"，不改標記，留待下次重試。
7. onPostExecute → reload()：上傳全部跑完後切回主 UI 執行緒，用 loadPending() 重新刷新待上傳列表，並向使用者顯示「成功上傳 N 筆」的 Toast。

核心程式

```
ArrayList<CloudPendingItem> list = db.getAllPending(true); // uploaded=0
for (CloudPendingItem item : list){
    HttpURLConnection conn = (HttpURLConnection) new URL(apiUrl).openConnection();
    conn.setRequestMethod("POST");
    conn.setDoOutput(true);
    // 1) 組 x-www-form-urlencoded
    String data =
        "name="+enc(item.getParticipantName())+
        "&exercise_weight="+item.getExerciseWeight()+
        // …(省略其餘欄位)…
        "&date="+enc(item.getExerciseDate());
    // 2) 傳送
    try(OutputStream os = conn.getOutputStream()){
        os.write(data.getBytes());
    }
    // 3) 讀回應 & 判斷
    int code = conn.getResponseCode();
    if(code==200 && readBody(conn).contains("success"))
        db.markUploaded(item.getId());      // ← 改 uploaded=1
    conn.disconnect();
}
```

圖 26 手機上傳核心程式碼

參數對照表

表 7 傳入雲端的資料表

POST 欄位	來源屬性	備註
name	getParticipantName()	URL encoded
height / weight / age	getHeightCm() …	整數 / 小數皆可
exercise_name	getExerciseName()	curl / raise / press
exercise_weight	getExerciseWeight()	kg
sets / reps	getSets() / getReps()	
total_time	getTotalSeconds()	sec

fatigue_sec	getFatigueSeconds()	允許空字串
date	getExerciseDate()	YYYY-MM-DD

錯誤處理&使用者回饋

上傳結果：onPostExecute() 顯示「成功上傳N筆」，並重新呼叫 loadPending()，即時刷新 ListView。

失敗策略：若 HTTP錯誤或字串不含 success，該筆紀錄仍留在 uploaded=0 佇列，使用者可稍後重試。

手動刪除：長按 ListView → AlertDialog，直接從 DB 刪除單筆。

伺服器端上傳介面

```php
// 1. 連線資料庫
$conn = new mysqli("localhost", "root", "", "fitness_cloud");
if ($conn->connect_error) {
    http_response_code(500);
    die("❌ DB fail: " . $conn->connect_error);
}

// 2. 驗證必要欄位（姓名、動作、日期）
if (!isset($_POST['name'], $_POST['exercise_name'], $_POST['date'])) {
    http_response_code(400);
    die("❗ 缺少必要欄位");
}

// 3. 取 POST 資料（允許空值）
$name   = $_POST['name'] ?? '';
$height = $_POST['height'] ?? 0;
$fatigue = ($_POST['fatigue_sec'] ?? '') === '' ? null : $_POST['fatigue_sec'];
// 其餘欄位省略…

// 4. 預備語句寫入資料庫
$stmt = $conn->prepare("INSERT INTO cloud_data (...) VALUES (?,?,?,?,...?,?)");
$stmt->bind_param("sdd...s", $name, $height, ..., $fatigue, $date);

// 5. 成功則回應 success，否則回應錯誤訊息
if ($stmt->execute()) {
    echo "✅ success";
} else {
    http_response_code(500);
    echo "❌ 上傳失敗: " . $stmt->error;
}
```

圖 27 伺服器上傳介面程式碼

1. 連接資料庫，若失敗回傳 HTTP 500。
2. 使用 isset() 驗證必要欄位是否齊全。
3. 對每個欄位加上 ?? 預設值，避免未傳入時導致錯誤。
4. 使用 prepared statement 防止 SQL injection。
5. 回傳結果供 App 端根據是否包含 "success" 判斷是否標記上傳成功。

(九)App 整體運作流程圖與架構圖

高階系統架構圖

展示 App、BLE、Arduino、本地與雲端之間的整體資料流程。

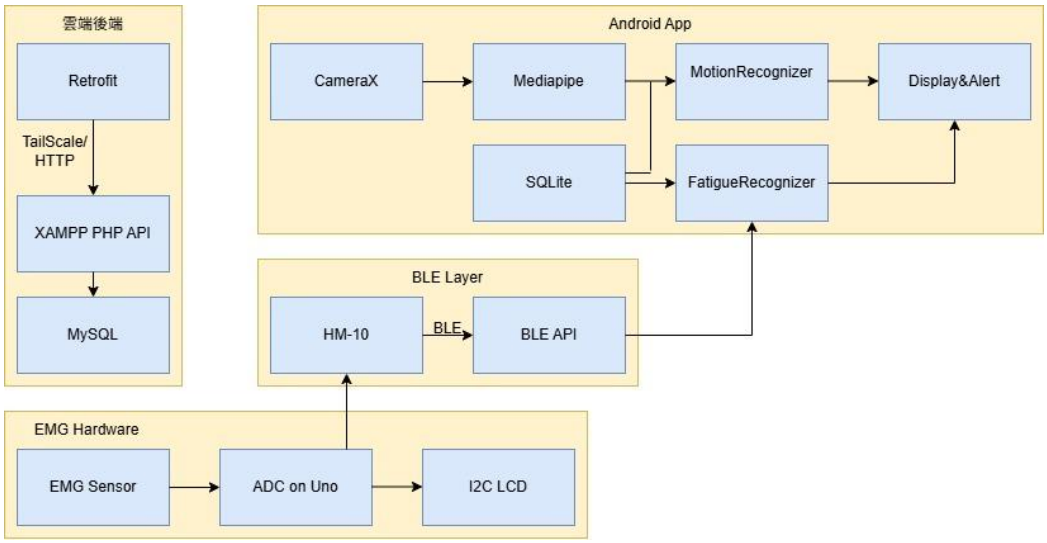


圖 28 高階系統架構圖

影像辨識模組架構圖

聚焦 CameraX 到 Mediapipe 到 PoseOverlayView 到 MotionRecognizer 的處理流程。

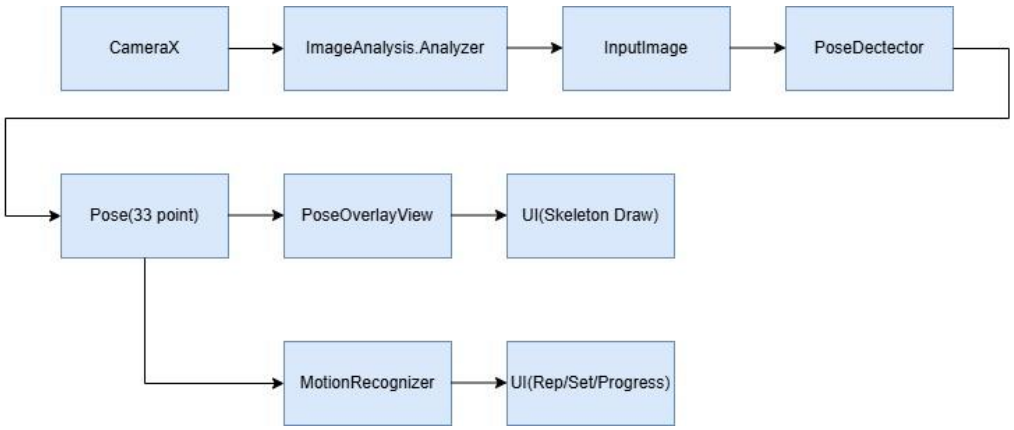


圖 29 影像辨識模組架構圖

疲勞偵測模組架構

顯示 ArduinoADC 到 HM-10 BLE 到手機端的 BLE API 到 RMS 演算法到警示邏輯的細節。

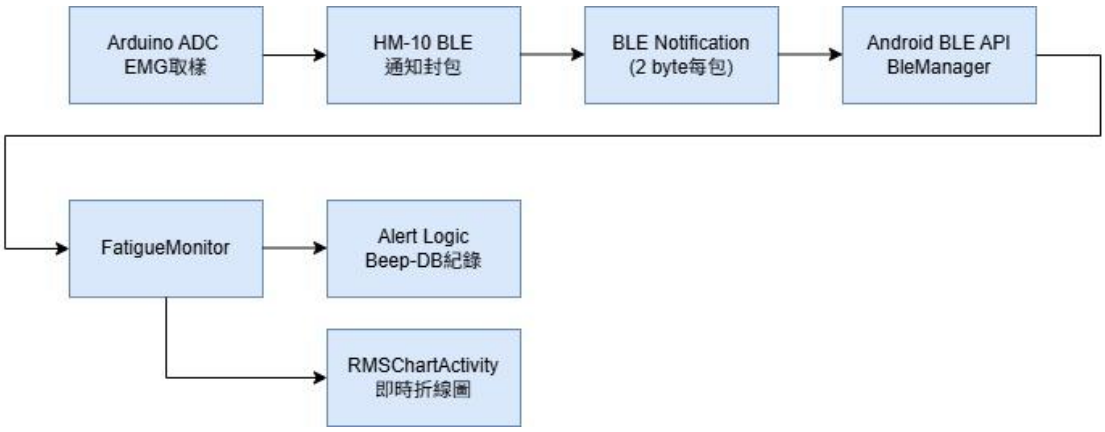


圖 30 疲勞偵測模組架構圖

雲平台連線架構圖

說明 XAMPP(Apache+PHP)+MySQL 與 App 經由 Tailscale 存取的拓樸。

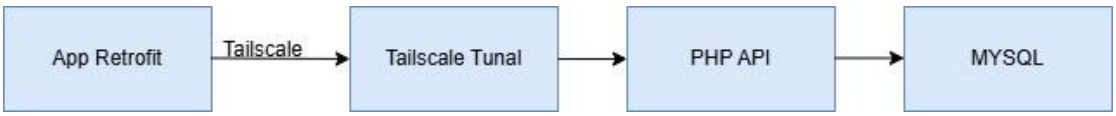


圖 31 雲平台架構圖

使用者流程圖

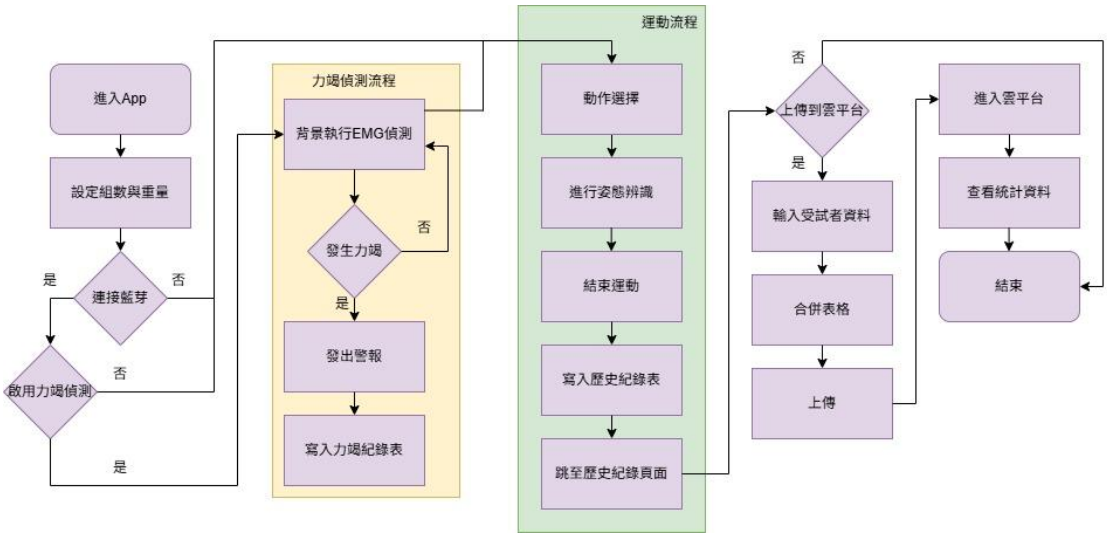


圖 32 使用者流程圖

四、 軟硬體分析

表 8 軟硬體分析圖表

類別	工具/元件	腳色與功能
IDE & Build	Android Studio, Gradle	開發、編譯、相依管理
開發語言	Java, Android SDK, C, html	UI、CameraX、BLE、SQLite、RMS 演算法
相機模組	CameraX, ML Kit Pose	實時擷取影像並偵測關鍵點
BLE 通訊	Android BLE API, HM-10	EMG 資料推播
資料庫	SQLiteOpenHelper, MySQL	本地儲存、雲端後端
圖表套件	MPAndroidChart	EMG & 運動次數折線圖
硬體感測	Arduino Uno,Grove EMG	EMG 讀取、現場顯示

自訂元件	PoseOverlayView, MotionRecognizer	骨架繪製、動作計數
後端技術	PHP, MySQL, Retrofit	API 架構、資料上傳
其他硬體元件	I²C LCD	方便開發時檢測數據

(一)軟體分析:

開發環境與工具

IDE／語言：Android Studio + Java 11

相依套件：

CameraX（androidx.camera:camera-core）

ML Kit Pose (com.google.mlkit:pose-detection)

MPAndroidChart (com.github.PhilJay:MPAndroidChart)

後端：XAMPP（Apache 2.4 + PHP 7.4 + MySQL 8）

系統架構與模組

手機 APP UI 層：(所有頁面都幾乎具備返回上一頁或首頁的功能)

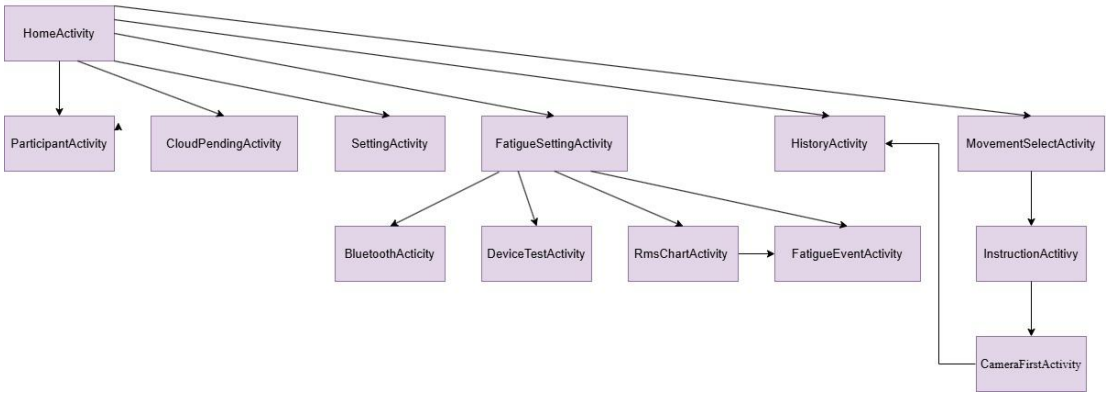


圖 33 手機 App UI 進入流程

表 9 手機 App UI 對應表格

UI 程式 / 功能	圖片

<table>
<tr>
<td>HomeActivity /
手機 App 得主頁面，負責進入 ParticipantActivity 、 CloudPendingActivity 、 SettingActivity 、 MovementSelectActivity 、 HistoryActivity、FatigueSettingActivity</td>
<td>

圖 34 HomeActivity</td>
</tr>
<tr>
<td>ParticipantActivity /
負責輸入受試者資料及瀏覽和管理受試者表的頁面(新增、刪除)</td>
<td>

圖 35 ParticipantActivity</td>
</tr>
</table>

CloudPendingActivity / 資料表整合為要上雲端的資料表並檢視與上傳	 圖 36 CloudPendingActivity
SettingActivity / 設定運動要做的重量、組數、組間休息時間	 圖 37 SettingActivity

MovementSelectActivity / 在運動開始前選擇要做的動作。例如：二頭彎舉、側平舉、肩推；進入 InstructionActitivy 並傳入動作參數	 圖 38 MovementSelectActivity
InstructionActitivy / 健身動作指導頁面，在運動開始前指導使用者做出正確的運動姿勢、流程，避免傷害；負責進入 CameraFirstActivity 並傳入動作參數	 圖 39 InstructionActitivy

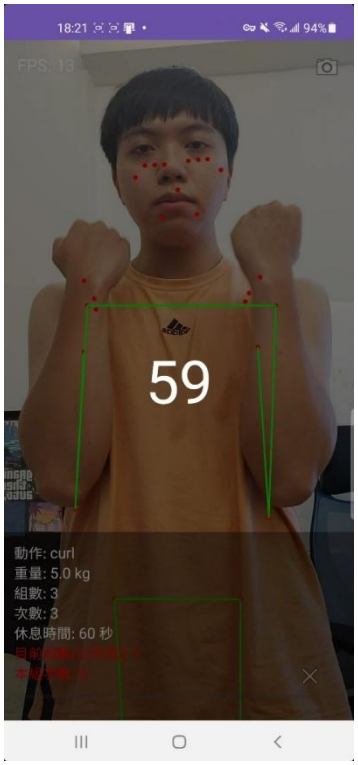
<table>
<tr>
<td>CameraFirstActivity /
影像辨識程式；運動結束進入 HistoryActivity</td>
<td> 

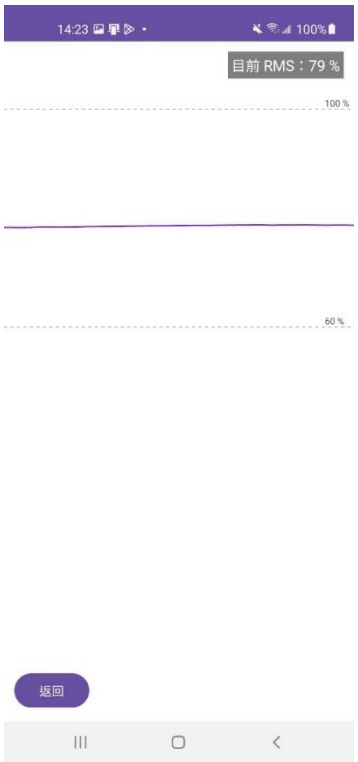
圖 40 CameraFirstActivity</td>
</tr>
<tr>
<td>HistoryActivity /
依行事曆查詢運動紀錄、管理紀錄(刪除)</td>
<td>

圖 41 HistoryActivity</td>
</tr>
</table>

<table>
<tr>
<td>FatigueSettingActivity /
一切與力竭裝置相關的 UI 頁面的進入；開啟/關閉力竭警報；負責進入 BluetoothActicity、DeviceTestActivity 、 FatigueEventActivity 、RmsChartActivity</td>
<td>

圖 42 FatigueSettingActivity</td>
</tr>
<tr>
<td>BluetoothActicity /
管理藍芽連線：掃描裝置、連線、中斷連線</td>
<td>

圖 43 BluetoothActicity</td>
</tr>
</table>

DeviceTestActivity / 圖表化的方式查看肌肉電訊號及用終端機兩種方式查看，以確認手機有收到藍芽傳輸過來的資料	 圖 44 DeviceTestActivity
RmsChartActivity / 繪製 EMG 歸一化 RMS 數據曲線圖，提供使用者即時或事後的強度變化趨勢分析	 圖 45 RmsChartActivity

1. 影像偵測模組：

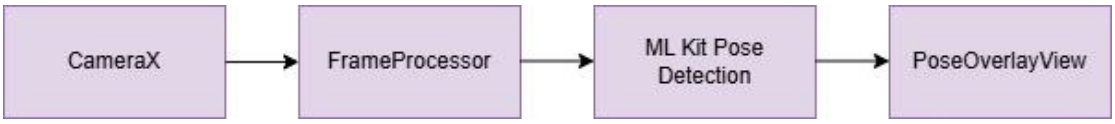


圖 46 影像偵測模組流程圖

1. CameraX

   「鏡頭入口」，負責開啟相機、抓取畫面並顯示在螢幕上，同時把最新一幀影像交給後續處理。

2. FrameProcessor

   中繼站：

   - 先把 CameraX 給的 ImageProxy（YUV 格式）轉成 ML Kit 能讀的 InputImage（RGB + 旋轉資訊）
   - 再把這張轉好的影像丟給 Pose Detector，自己不做判斷，只負責「包裝＋轉交」。

3. ML Kit Pose Detection

   真正「動腦筋」的地方：它拿到 InputImage 後跑模型，瞬間找出人體的 33 個關鍵點（肩膀、手肘、手腕…），然後回傳一個 Pose 物件。

4. PoseOverlayView

   拿到 Pose 之後，就負責把那些點畫到畫面上：

   - 用紅點標記每個關鍵點
   - 用綠線把肩–肘–腕、髖–膝–踝依照身體骨架連起來
   - 如果是前鏡頭，還會自動做左右翻轉校正，讓畫面畫得跟鏡子一樣自然。

2. 動作計數模組：MotionRecognizer（三點夾角演算法）
3. EMG 處理模組：

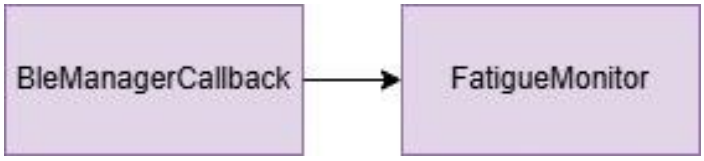


圖 47 EMG 處理模組流程圖

1. BleManagerCallback
   - 這是 BleManager 在收到 HM-10 推送過來的原始 byte[] EMG 資料時，會呼叫的介面方法（onCharacteristicChanged() → onDataReceived()）。
   - 它像一個「資料門神」，負責把 BLE stack 的原始 EMG 值傳遞給下一個處理模組，讓後面可以直接拿到整串肌電訊號。
2. FatigueMonitor
   - 拿到 BleManagerCallback 傳來的 EMG 數值後，開始做 MVC 基線建構（前 100 筆取最大值），再用 200 樣本滑動視窗 算 RMS，並做歸一化（norm = RMS / MVC）。

- 當連續多次 norm 低於 0.6(肌肉高負荷)就判定「力竭」→ 播嗶聲、寫入 SQLite 力竭事件，並透過 listener 通知 UI 更新。
- 同時也不斷把最新的 norm 值透過 listener 回報給畫面，讓你看到即時的肌電強度變化。

4. 資料存取層：

本地：SQLite + DAO

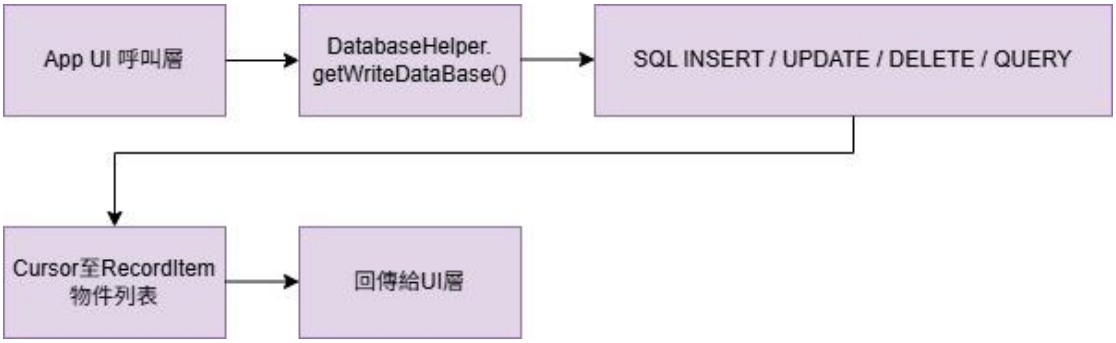


圖 48 本地資料存取

1. App UI 層呼叫：HistoryActivity、CameraFirstActivity、FatigueMonitor 等需要存取時，呼叫 DatabaseHelper 的方法（如 getRecordsByDate()、insertRecord()、insertFatigueEvent()）。
2. getWritableDatabase()：打開或建立本地 SQLite 資料庫，準備寫入或讀取。
3. SQL 操作：執行 INSERT、UPDATE、DELETE 或 SELECT。
4. Cursor → 物件列表：SELECT 查詢返回一個 Cursor，迭代之後封裝成 RecordItem、FatigueEvent 等物件。
5. 回傳給 UI 層：把物件列表傳給 RecyclerView Adapter 或 ViewModel，畫面上就能顯示歷史記錄或即時力竭清單。

雲端：

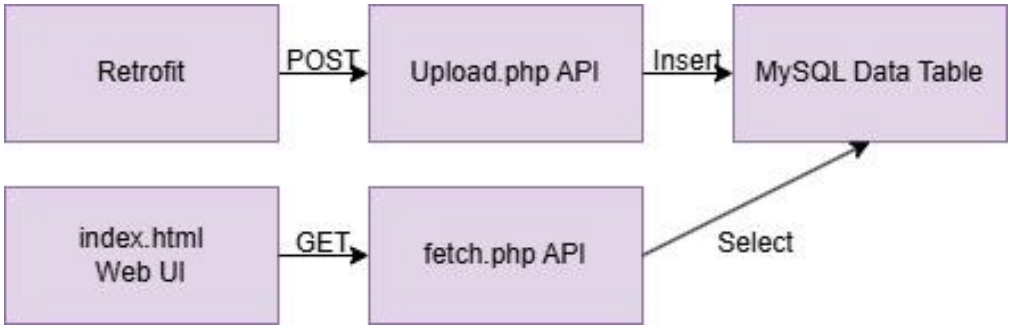


圖 49 雲端資料存取

1. Retrofit Client：Android 端呼叫 upload.php、fetch.php。
2. upload.php API：接收 POST，寫入 MySQL。
3. fetch.php API：回傳查詢結果給 Web UI。
4. index.html (Web UI)：用 GET 拿資料並顯示圖表。

5. 雲平台示意圖：

受試者清單

Show 10 entries　　Search:

姓名	筆數	最後日期
A001	1	2025-05-08
A002	1	2025-05-08
A003	1	2025-05-08
A004	1	2025-05-08
A005	1	2025-05-08
Jay	3	2025-05-29
jay	1	2025-05-08
Lisa	1	2025-05-10
test1	1	2025-05-09

Showing 1 to 9 of 9 entries　　Previous 1 Next

圖 50 雲平台選取受試者

Jay

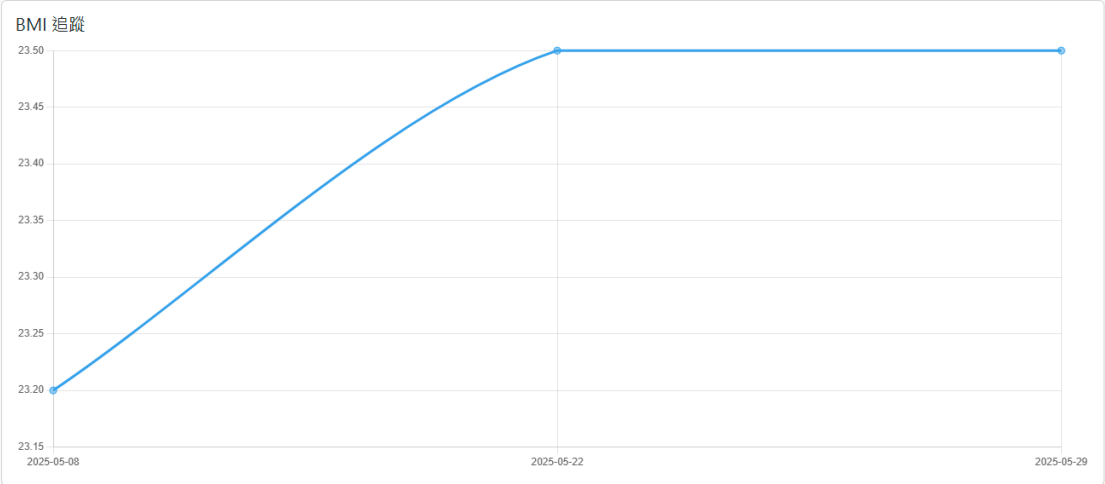


圖 51 雲平台 BMI 曲線

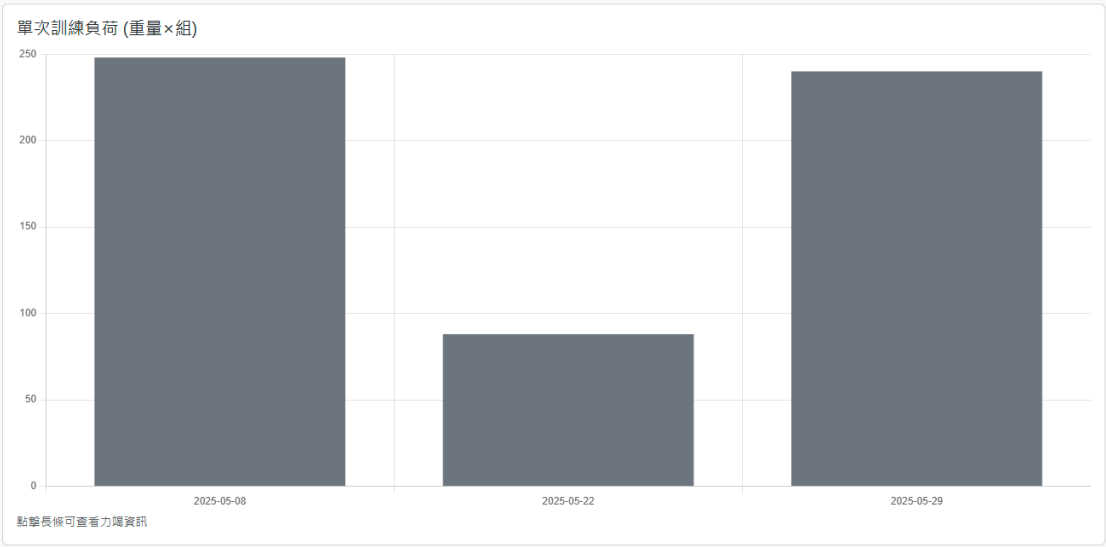


圖 52 雲平台訓練容量

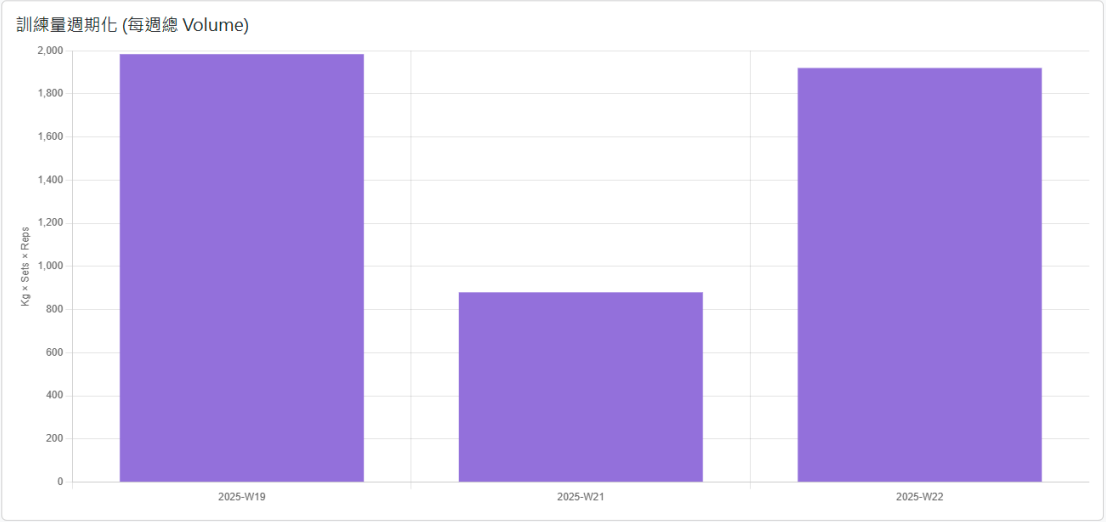
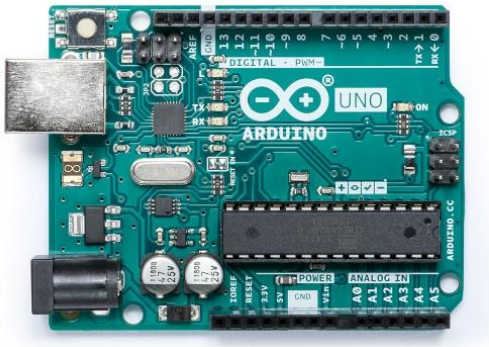
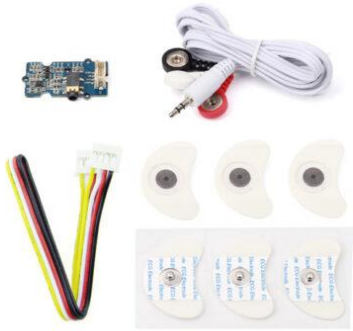


圖 53 雲平台訓練週期

(二)硬體／電路分析

主要元件清單

元件 / 功能說明	圖片

Arduino Uno 3 / ADC 讀取、UART 控制 HM-10、I²C LCD 驅動	 圖 54 Arduino Uno 3
Grove EMG Detector / 偵測肌電電位（0–5V 輸出）	 圖 55 Grove EMG Detector
HM-10 / BLE 通訊，UUID 0xFFE0/0xFFE1	 圖 56 HM-10
I2C LCD 1602a / 顯示「EMG 值」、「系統狀態」	 圖 57 2C LCD 1602a

Android 9.0 手機 / CameraX、BLE API、SQLite 運行環境	 圖 58 samsung galaxy A60

表 10 硬體元件對照表

電路示意圖

ADC 連接：EMG 模組 OUT → Uno A0

BLE：Uno TX/RX ↔ HM-10 UART

LCD：I²C SDA/SCL ↔ Uno A4/A5

電源：5V/3.3V 標註、共地(GND)

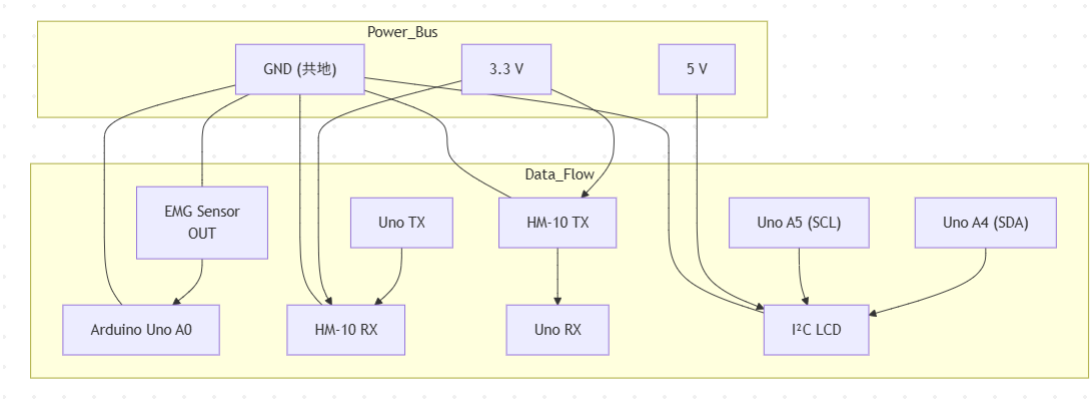


圖 59 電路示意圖

資料與電力流程

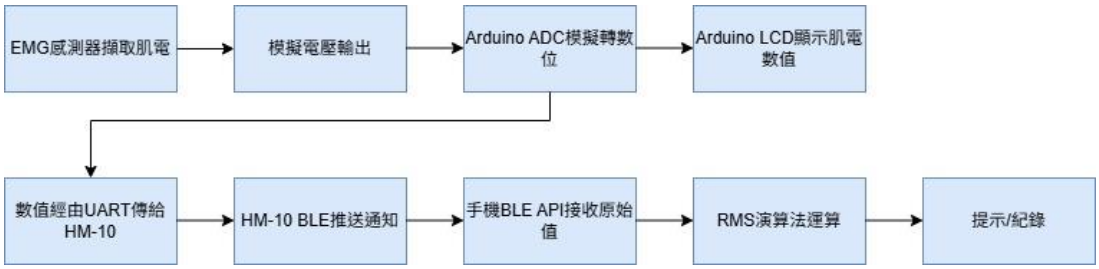


圖 60 資料與電力流程

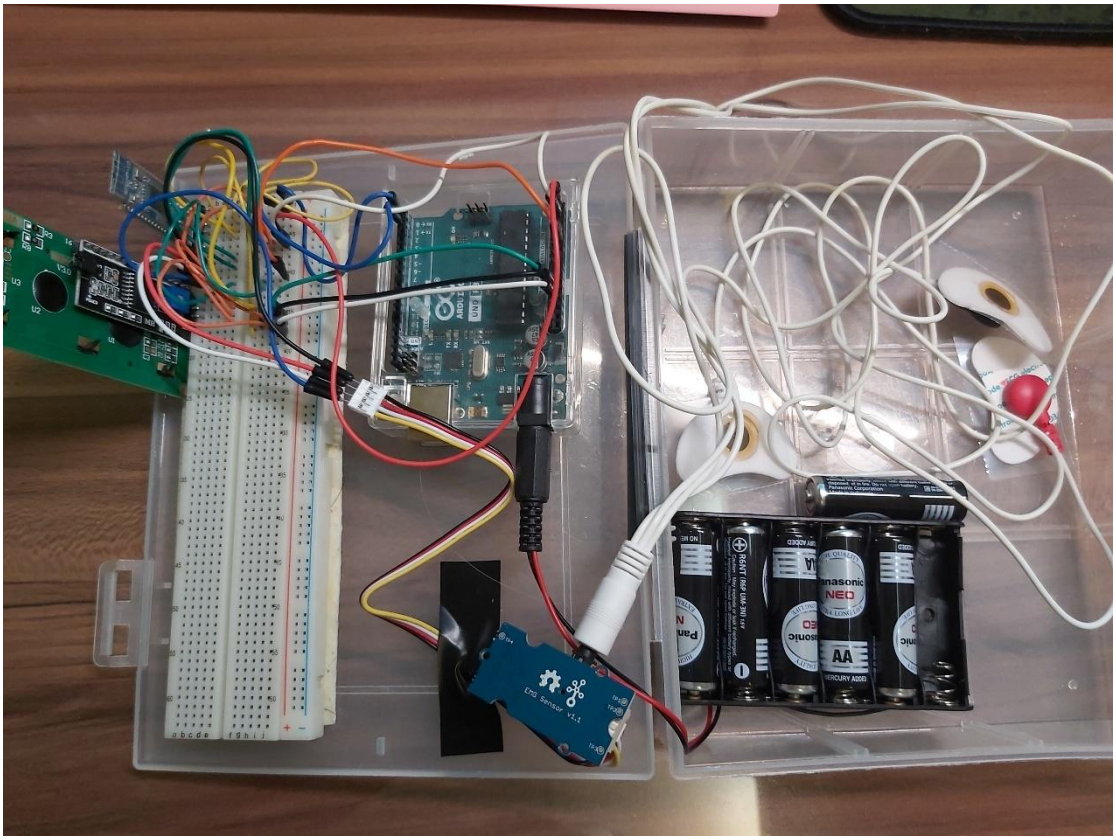


圖 61 硬體元件實作展示

五、 研究結果

主觀滿意度問卷（Likert 5 分量表）

- 力竭警示準確度：3.8 分（76 % 滿意度）
- 動作計數準確度：4.6 分（92 % 滿意度）
- 整體使用便利性：4.6 分（92 % 滿意度）

本研究階段共招募 5 位受試者（男 3 女 2，年齡 20－23 歲）進行系統測試，並結合「主觀滿意度問卷」、「使用者操作紀錄」與「教練回饋」多角度評估系統效能：

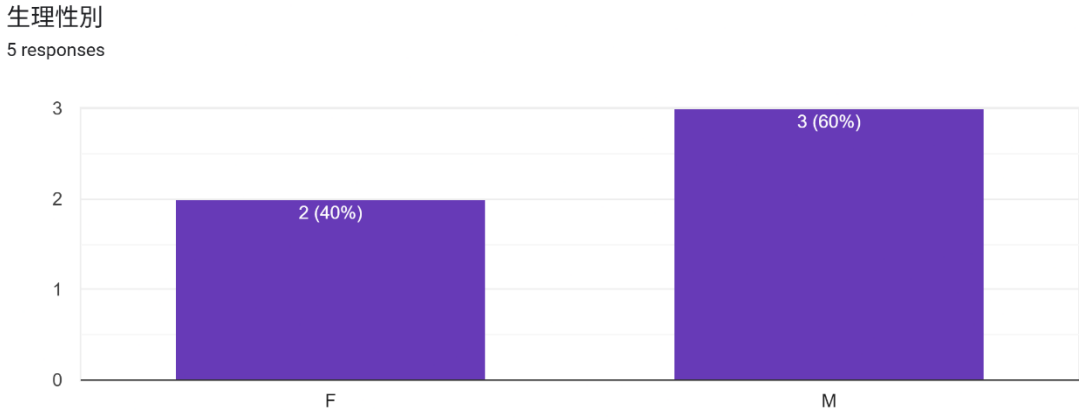


圖 62 滿意度調查生理性別

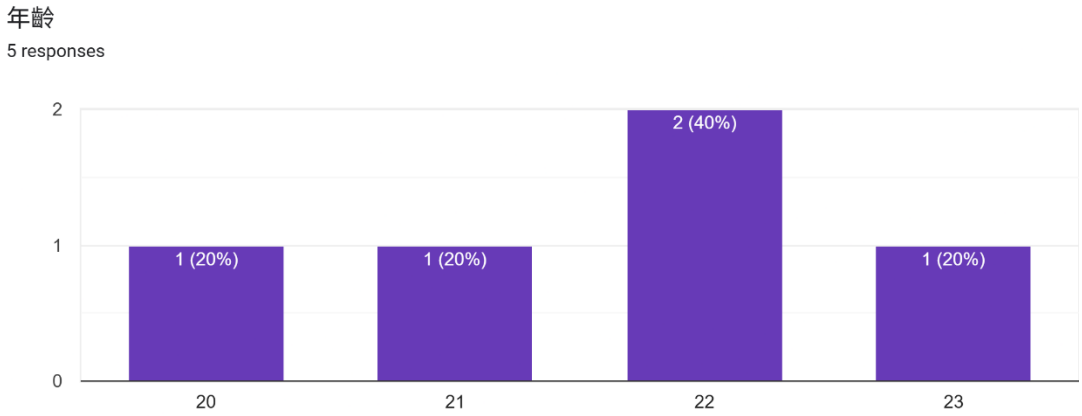


圖 63 滿意度調查受試者年齡

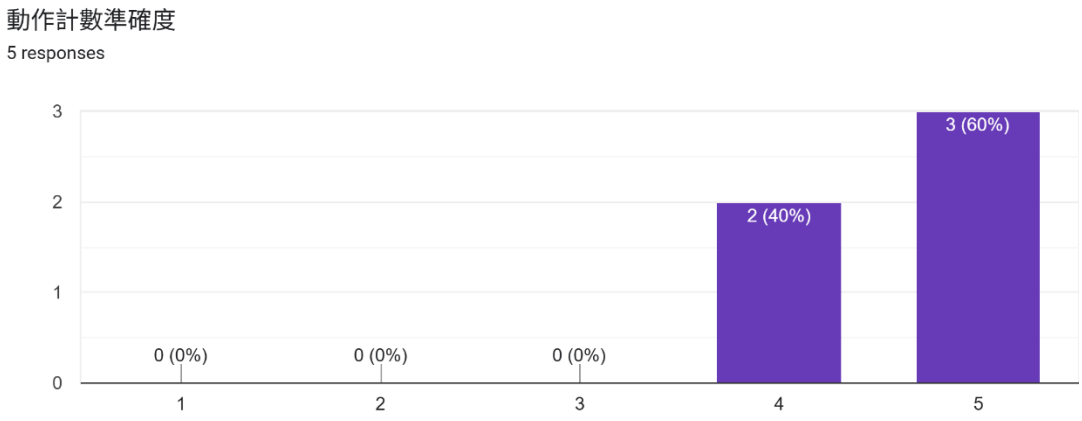


圖 64 滿意度調查動作精準度

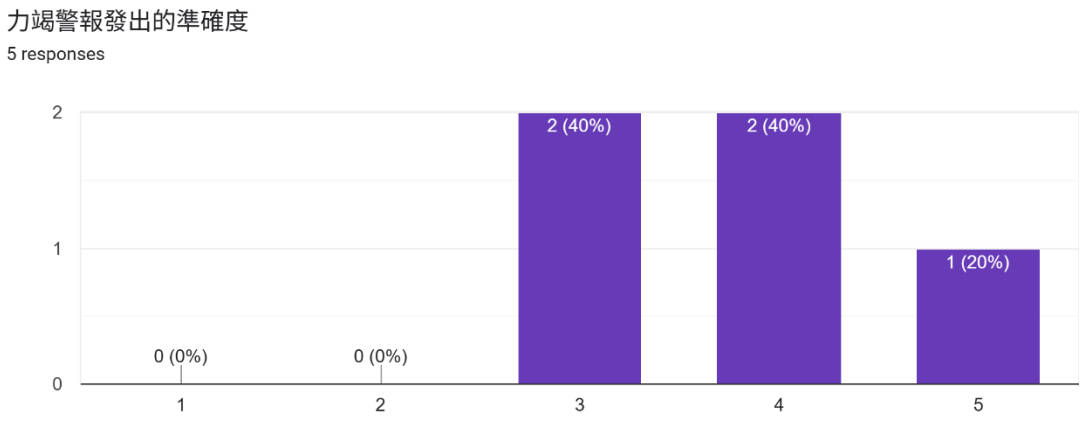


圖 65 滿意度調查力竭警報精準度

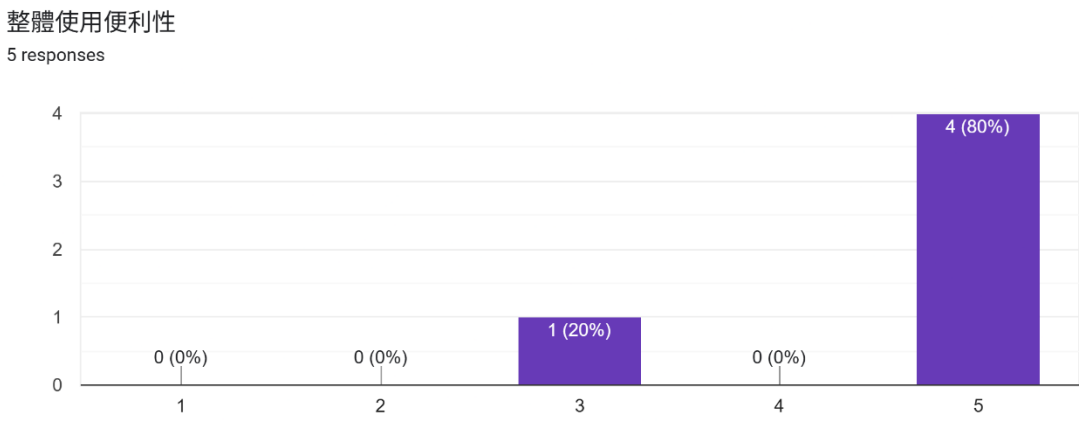


圖 66 滿意度調查整體使用便利性

自動化動作計數效果

系統自動記錄每一動作組數約 10 組，與市面上的健身 App 進行比較，手機畫面點擊次數少 30 次 ，大幅提升運動流暢度。

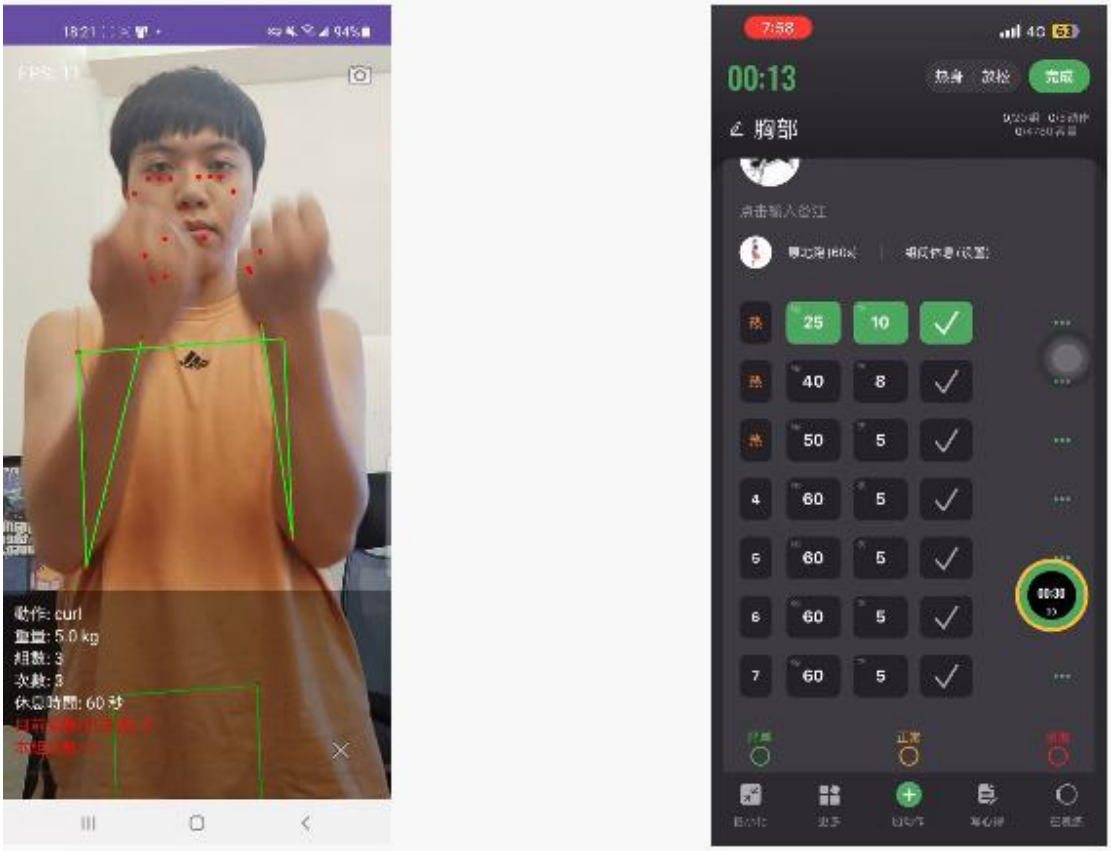


圖 67 與市面上的 App 比較

專業教練回饋

健身教練試用後均肯定：「RMS 警示時間與實際肌肉疲勞狀態高度吻合，有助於初學者安全訓練。」

圖 68 教練使用裝置實測

六、 結論

本專題成功開發並驗證一套端到端的智慧健身輔助系統，整合即時影像姿勢偵測（CameraX + MediaPipe）、肌電疲勞判定（RMS，滿意度 76 %）、BLE 通訊與雲端資料管理。實驗顯示，自動計數平均誤差 ±1 reps(動作需完整到位，否則可能會少一次計數)，力竭警示約提前 2 s 提醒休息；專業教練亦肯定其安全輔助價值。

展望未來，本專題將沿以下方向深化與擴展：首先，透過更大規模的多族群受試調查與重複量測，提升統計信度並驗證演算法的泛化能力；其次，導入個人化閾值校正與多維肌電特徵，借助機器學習／深度學習模型（如 SVM、LSTM）進行疲勞偵測，預期能將辨識準確度推升至 85% 以上；此外，計畫擴充系統支援更多動作模式（如深蹲、硬舉）並開發跨平台（iOS、Web）版本，以擴大適用範圍；最後，將與健身房及復健中心合作，進行長期臨床驗證與商業化測試，評估系統在運動傷害預防與復健輔助上的實際效益。

七、 參考文獻

[1] Cao, Z., Simon, T., Wei, S.-E., & Sheikh, Y. (2017). *Realtime Multi-Person 2D Pose Estimation using Part Affinity Fields*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 7291–7299.

[2] Bazarevsky, V., Kartashov, I., Grishchenko, I., & Grundmann, M. (2020). *BlazePose: On-device Real-time Body Pose Tracking*. arXiv:2006.10204.

[3] De Luca, C. J. (1997). *The use of surface electromyography in biomechanics*. Journal of Applied Biomechanics, 13(2), 135–163.

[4] Bluetooth Special Interest Group. (2016). *Bluetooth Core Specification Version 5.0*. Bluetooth SIG.

[5] Google. (2021). *ML Kit for Firebase: Pose detection*. Retrieved from https://developers.google.com/ml-kit/vision/pose-detection

[6] Google. (2020). *CameraX Jetpack library*. Android Developers. Retrieved from https://developer.android.com/training/camerax

[7] PhilJay. (2018). *MPAndroidChart: A powerful & easy to use chart library for Android*. GitHub repository. Retrieved from https://github.com/PhilJay/MPAndroidChart

[8] Solomonow, M., Baratta, R., Banks, A., & Bennett, B. (1987). *Electromyogram power spectrum frequencies associated with motor-unit recruitment strategies*. Journal of Applied Physiology, 63(6), 2351–2359.

[9] Winter, D. A. (2009). *Biomechanics and Motor Control of Human Movement* (4th ed.). Wiley.

[10] SQLite Consortium. (2021). *SQLite Documentation*. Retrieved from https://www.sqlite.org/docs.html

[11] Youtube: AI Pose Estimation with Python and MediaPipe | Plus AI Gym Tracker Project